Università della Svizzera italiana

Faculty of Informatics

Machine Learning

Homework Report #2

Baio Federico

26 October 2025

1 Calculating Gradients

Given a two layer neural network defined by the following equation:

$$y := \mathbf{w}^{(2)\top} f(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + b^{(2)}$$

to simplify the notation we may want to write:

$$\mathbf{u}^{(1)} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{z}^{(1)} = f(\mathbf{u}^{(1)})$$

the exercise ask us to evaluate (numerically) the following loss function

$$L = \frac{1}{2}\sum_{k=1}^{3}(y_k - t_k)^2$$

as well as the next gradients:

$$\frac{\partial L}{\partial W^{(1)}}, \quad \frac{\partial L}{\partial \mathbf{w}^{(2)}}, \quad \frac{\partial L}{\partial \mathbf{b}^{(1)}}, \quad \frac{\partial L}{\partial b^{(2)}}$$

1.1 Partial derivative solved analytically

So as a first thing we start by solving the partial derivative analytically, then we plug in the matrices and vectors to get the numerical values. A clever thing to do is solving them starting from the most external layer and use the chain rule[1].

1.1.1 Solution of $\frac{\partial L}{\partial b^{(2)}}$

As a first thing, using the chain rule, we write the partial derivative as:

$$\frac{\partial L}{\partial b^{(2)}} = \sum_{k=1}^{3} \frac{\partial L}{\partial y_k}\frac{\partial y_k}{\partial b^{(2)}}$$

in this way we obtain:

$$\frac{\partial L}{\partial y_k} = (y_k - t_k)$$

and:

$$\frac{\partial y_k}{\partial b^{(2)}} = \frac{\partial}{\partial b^{(2)}}(\mathbf{w}^{(2)\top}\mathbf{z}^{(1)} + b^{(2)}) = 1$$

where we leave the first part in vector form to not complicate the solution since is seen as a constant. So we ends up with:

$$\frac{\partial L}{\partial b^{(2)}} = \sum_{k=1}^{3} \frac{\partial L}{\partial y_k}\frac{\partial y_k}{\partial b^{(2)}} = \sum_{k=1}^{3}(y_k - t_k)$$

[1]Note: To keep the notation lighter, I'll omit the explicit dependence on the sample index k (e.g. writing $z_i^{(1)}$ instead of $z_{i,k}^{(1)}$). It is trivial that each time that we use the chain rule to take the partial derivative of y_k (or of an element inside y_k), we'll have dependence on k

1.1.2 Solution of $\frac{\partial L}{\partial \mathbf{w}^{(2)}}$

We do the same thing here. Applying the chain rule and writing the partial derivatives element-wise we obtain:

$$\frac{\partial L}{\partial \mathbf{w}^{(2)}} = \sum_{k=1}^{3} \frac{\partial L}{\partial y_k} \frac{\partial y_k}{\partial \mathbf{w}^{(2)}} = \sum_{k=1}^{3} \frac{\partial L}{\partial y_k} \frac{\partial y_k}{\partial w_i^{(2)}}$$

with $\frac{\partial L}{\partial y_k}$ already evaluated above and writing element-wise:

$$\frac{\partial y_k}{\partial w_i^{(2)}} = \frac{\partial}{\partial w_i^{(2)}} \left(\sum_{p=1}^{2} w_p^{(2)} z_p^{(1)} + b^{(2)} \right) = \sum_{p=1}^{2} z_p^{(1)} \delta_{p,i} = z_i^{(1)}$$

where the Kronecker delta tells us that is 0 for $p \neq i$ so we can remove the sum and we leave z_i instead of expanding it just out of simplicity because the value $w_i^{(2)}$ is not contained inside and we treat it like a multiplier constant. So as we before we can, using the chain rule writing:

$$\frac{\partial L}{\partial w_i^{(2)}} = \sum_{k=1}^{3} \frac{\partial L}{\partial y_k} \frac{\partial y_k}{\partial w_i^{(2)}} = \sum_{k=1}^{3} \left((y_k - t_k) z_i^{(1)} \right)$$

1.1.3 Solution of $\frac{\partial L}{\partial \mathbf{b}^{(1)}}$

In this case the solution becomes more complicated since the value of $\mathbf{b}^{(1)}$ is inside the previous layer. With the chain rule we can say that (solving each partial derivative element-wise):

$$\frac{\partial L}{\partial \mathbf{b}^{(1)}} = \sum_{k=1}^{3} \sum_{p=1}^{2} \sum_{r=1}^{4} \frac{\partial L}{\partial y_k} \frac{\partial y_k}{\partial z_p^{(1)}} \frac{\partial z_p^{(1)}}{\partial u_r^{(1)}} \frac{\partial u_r^{(1)}}{\partial b_i^{(1)}}$$

so, we now solve each partial derivative starting from the left ($\frac{\partial L}{\partial y_k}$ as already been solved)

$$\frac{\partial y_k}{\partial z_p^{(1)}} = \frac{\partial}{\partial z_p^{(1)}} \left(\sum_{s=1}^{2} w_s^{(2)} z_s^{(1)} + b^{(2)} \right) = w_p^{(2)}$$

where the sum in s cancels out because in any other case besides $s = p$ the sum is zero. The next derivative is:

$$\frac{\partial z_p^{(1)}}{\partial u_r^{(1)}} = \frac{\partial}{\partial u_r^{(1)}} \left(f(u_p^{(1)}) \right) = \mathbb{1}_{\{u_p^{(1)} > 0\}} \delta_{p,r}$$

which is equal to the indicator function since the function f is defined arbitrary as the ReLU and since is applied element-wise we write $\delta_{p,r}$ since gives 0 when $z_p^{(1)}$ does not depend on $u_r^{(1)}$ (namely $p \neq r$). The last partial derivative is trivially:

$$\frac{\partial u_r^{(1)}}{\partial b_i^{(1)}} = \frac{\partial}{\partial b_i^{(1)}} \left(\sum_{m=1}^{4} w_{r,m}^{(1)} x_m + b_r^{(1)} \right) = \delta_{r,i}$$

So, summarizing, we have that:

$$\frac{\partial L}{\partial b_i^{(1)}} = \sum_{k=1}^{3} \sum_{p=1}^{2} \sum_{r=1}^{4} (y_k - t_k) w_p^{(2)} \mathbb{1}_{\{u_r^{(1)} > 0\}} \delta_{p,r} \delta_{r,i} = \sum_{k=1}^{3} (y_k - t_k) w_i^{(2)} \mathbb{1}_{\{u_i^{(1)} > 0\}}$$

where because of the Kronecker delta we remove the two summation.

1.1.4 Solution of $\frac{\partial L}{\partial W^{(1)}}$

The last one is the most complicated but most of the partial derivative as already been solved so remain to solve just the last one:

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \sum_{k=1}^{3}\sum_{p=1}^{2}\sum_{r=1}^{4} \frac{\partial L}{\partial y_k}\frac{\partial y_k}{\partial z_p^{(1)}}\frac{\partial z_p^{(1)}}{\partial u_r^{(1)}}\frac{\partial u_r^{(1)}}{\partial w_{i,j}^{(1)}}$$

that is simply:

$$\frac{\partial u_r^{(1)}}{\partial w_{i,j}^{(1)}} = \frac{\partial}{\partial w_{i,j}^{(1)}}\left(\sum_{m=1}^{4} w_{r,m}^{(1)} x_m + b_r^{(1)}\right) = \sum_{m=1}^{4} x_m \delta_{r,i}\delta_{m,j} = x_j \delta_{r,i}$$

since for any other value of r and m different different from i and j the partial derivative is simply 0. So the final solution is:

$$\frac{\partial L}{\partial w_{i,j}^{(1)}} = \sum_{k=1}^{3}\sum_{p=1}^{2}\sum_{r=1}^{4}(y_k - t_k) w_p^{(2)} \mathbb{1}_{\{u_r^{(1)}>0\}} \delta_{p,r} x_j \delta_{r,i} = \sum_{k=1}^{3}(y_k - t_k) w_i^{(2)} \mathbb{1}_{\{u_{i,k}^{(1)}>0\}} x_j$$

1.2 Numerical evaluation

We remind here the numerical values as indicated in the problem statement: the values of the 3 input vectors $\mathbf{x}_k \in \mathbb{R}^4$ (summarized in the matrix $\mathbf{X} \in \mathbb{R}^{4\times 3}$), targets t_k (summarized in the vector $\mathbf{t} \in \mathbb{R}^3$), weights $\mathbf{W}^{(1)} \in \mathbb{R}^{2\times 4}$ and $\mathbf{w}^{(2)} \in \mathbb{R}^2$ as well as biases $\mathbf{b}^{(1)} \in \mathbb{R}^2$ and $b^{(2)} \in \mathbb{R}$ are given below.

$$\mathbf{X} = \begin{bmatrix} | & | & | \\ \mathbf{x}_1 & \mathbf{x}_2 & \mathbf{x}_3 \\ | & | & | \end{bmatrix} = \begin{bmatrix} 0.3 & 0.9 & -0.6 \\ 0.2 & -0.7 & 0.5 \\ -0.5 & 0.3 & 0.9 \\ 0.5 & 0.1 & -0.8 \end{bmatrix}$$

$$\mathbf{t} = \begin{bmatrix} t_1 & t_2 & t_3 \end{bmatrix}^\top = \begin{bmatrix} 0.6 \\ 0.2 \\ -0.7 \end{bmatrix}$$

$$\mathbf{W}^{(1)} = \begin{bmatrix} 0.6 & 0.2 & -0.5 & -0.3 \\ 0.5 & -0.4 & 1.0 & 0.2 \end{bmatrix} \quad \mathbf{b}^{(1)} = \begin{bmatrix} -0.2 \\ -0.6 \end{bmatrix}$$

$$\mathbf{w}^{(2)} = \begin{bmatrix} 0.4 \\ 0.7 \end{bmatrix} \quad b^{(2)} = -0.5$$

The function f is the **ReLU** nonlinearity, applied element-wise.

1.2.1 Evaluation of the Loss function

We start by evaluating the loss function. To get the matrix $\mathbf{Y}$ that contains the 3 values for each $\mathbf{x}_k$ we first need the matrix $\mathbf{U}^{(1)}$ of the first layer that include the all matrix $\mathbf{X}$:

$$\mathbf{U}^{(1)} = \mathbf{W}^{(1)}\mathbf{X} + b^{(1)}\mathbf{1}^\top$$

$$\mathbf{W}^{(1)}\mathbf{X} = \begin{pmatrix} 0.6 & 0.2 & -0.5 & -0.3 \\ 0.5 & -0.4 & 1.0 & 0.2 \end{pmatrix} \begin{pmatrix} 0.3 & 0.9 & -0.6 \\ 0.2 & -0.7 & 0.5 \\ -0.5 & 0.3 & 0.9 \\ 0.5 & 0.1 & -0.8 \end{pmatrix} = \begin{pmatrix} 0.32 & 0.22 & -0.47 \\ -0.33 & 1.05 & 0.24 \end{pmatrix}$$

We then add $b^{(1)} = [-0.2, -0.6]^\top$ on every column:

$$\mathbf{U}^{(1)} = \mathbf{W}^{(1)}\mathbf{X} + b^{(1)}\mathbf{1}^\top = \begin{pmatrix} 0.32 & 0.22 & -0.47 \\ -0.33 & 1.05 & 0.24 \end{pmatrix} + \begin{pmatrix} -0.2 & -0.2 & -0.2 \\ -0.6 & -0.6 & -0.6 \end{pmatrix} = \begin{pmatrix} 0.12 & 0.02 & -0.67 \\ -0.93 & 0.45 & -0.36 \end{pmatrix}$$

After that, we need to apply the non linearity on the matrix $\mathbf{U}^{(1)}$ using the ReLU function to get the matrix $\mathbf{Z}^{(1)}$ of the neurons for the layer 1:

$$\mathbf{Z}^{(1)} = \max(0, \mathbf{U}^{(1)}) = \begin{pmatrix} 0.12 & 0.02 & 0 \\ 0 & 0.45 & 0 \end{pmatrix}$$

Now that we have the value $\mathbf{Z}^{(1)} = f(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$ we can evaluate:

$$\mathbf{Y} = \mathbf{w}^{(2)}\mathbf{Z}^{(1)} + b^{(2)}$$

$$\mathbf{Y} = \begin{pmatrix} 0.4 & 0.7 \end{pmatrix} \begin{pmatrix} 0.12 & 0.02 & 0 \\ 0 & 0.45 & 0 \end{pmatrix} + (-0.5) = \begin{pmatrix} -0.452 & -0.177 & -0.5 \end{pmatrix}$$

Finally, the Loss function is simply given by:

$$L = \frac{1}{2}\sum_{k=1}^{3}(y_k - t_k)^2 = 0.5[(-0.452 - 0.6)^2 + (-0.177 - 0.2)^2 + (-0.5 + 0.7)^2] = 0.6444$$

1.2.2 Evaluation of each partial derivative

We start with the trivial evaluation of the second layer partial derivatives's that we can solve easily using the matrices evaluated above (we re-introduce here the complete notation with the dependence on k to make the numerical evaluation more clear). The first one is

$$\frac{\partial L}{\partial b^{(2)}} = \sum_{k=1}^{3}(y_k - t_k) = -1.2290$$

The second one have values:

$$\frac{\partial L}{\partial w_1^{(2)}} = \sum_{k=1}^{3}\left((y_k - t_k)z_{1,k}^{(1)}\right) = -0.13378, \qquad \text{with} \qquad z_1^{(1)} = \begin{bmatrix} 0.12 & 0.02 & 0 \end{bmatrix}$$

and

$$\frac{\partial L}{\partial w_2^{(2)}} = \sum_{k=1}^{3}(y_k - t_k)z_{2,k}^{(1)} = -0.16965, \qquad \text{with} \qquad z_2^{(1)} = \begin{bmatrix} 0 & 0.45 & 0 \end{bmatrix}$$

So we have:

$$\frac{\partial L}{\partial \mathbf{w}^{(2)}} = \begin{bmatrix} -0.13378 & -0.16965 \end{bmatrix}^T$$

For the partial derivatives of the first layer the evaluation is more complicated since we need to take into account the derivative of the ReLU function. We start so by defining the matrix that contains the partial derivatives $\frac{\partial z_p^{(1)}}{\partial u_r^{(1)}}$:

$$\nabla_{\mathbf{U}^{(1)}} \mathbf{Z}^{(1)} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

So that:

- $\mathbb{1}_{\{u_1^{(1)}>0\}} = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix}$
- $\mathbb{1}_{\{u_2^{(1)}>0\}} = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}$

The values for the partial derivative of the first layer w.r.t. $b^{(1)}$ are:

$$\frac{\partial L}{\partial b_1^{(1)}} = \sum_{k=1}^{3} \left(y_k - t_k) w_1^{(2)} \mathbb{1}_{\{u_{1,k}^{(1)}>0\}} \right) = \sum_{k=1}^{3} \left((y_k - t_k) \cdot 0.4 \cdot \mathbb{1}_{\{u_{1,k}^{(1)}>0\}} \right) =$$

$$= \sum_{k=1}^{3} \begin{bmatrix} -0.4208 & -0.1508 & 0 \end{bmatrix} = -0.5716$$

and

$$\frac{\partial L}{\partial b_2^{(1)}} = \sum_{k=1}^{3} \left(y_k - t_k) w_2^{(2)} \mathbb{1}_{\{u_{2,k}^{(1)}>0\}} \right) = \sum_{k=1}^{3} \left((y_k - t_k) \cdot 0.7 \cdot \mathbb{1}_{\{u_{2,k}^{(1)}>0\}} \right) =$$

$$= \sum_{k=1}^{3} \begin{bmatrix} 0 & -0.2639 & 0 \end{bmatrix} = -0.2639$$

At the end, the third partial derivative is:

$$\frac{\partial L}{\partial \mathbf{b}^{(1)}} = \begin{bmatrix} -0.5716 & -0.2639 \end{bmatrix}^T$$

Then, the last one is more involved since we have the neuron i, and for each neuron the value of the feature j, we show as example the value for the partial derivative w.r.t. $w_{1,2}$ and then directly the matrix of $\frac{\partial L}{\partial W^{(1)}}$:

$$\frac{\partial L}{\partial w_{1,2}^{(1)}} = \sum_{k=1}^{3} (y_k - t_k) w_1^{(2)} \mathbb{1}_{\{u_{1,k}^{(1)}>0\}} x_{2,k} = \sum_{k=1}^{3} (y_k - t_k) \cdot 0.4 \cdot \mathbb{1}_{\{u_{1,k}^{(1)}>0\}} \cdot x_{2,k} = 0.0214$$

with

$$x_{2,k} = \begin{bmatrix} 0.2 & -0.7 & 0.5 \end{bmatrix}$$

So the final matrix is equal to:

$$\frac{\partial L}{\partial W^{(1)}} = \begin{bmatrix} -0.26196 & 0.0214 & 0.16516 & -0.22548 \\ -0.23751 & 0.18473 & -0.07917 & -0.02639 \end{bmatrix}$$

2 Gradient Descent

We need to find the values of η that guarantees that the gradient descent for any starting point will converge to the minimum value of the function:

$$f(x) = 3x^2 + 4x + 1$$

To make sure that we are reaching the minimum, the error (difference between real minimum of the function) should decrease at each iteration. So, we need a function that connect the two errors at consecutive iterations (e_{n+1} and e_n) and that contains the value of η. We first of all find the value of the derivative of f:

$$f'(x) = 6x + 4$$

and derive the real minimum of this parabola, that is:

$$f'(x) = 0 \longleftrightarrow 6x + 4 = 0 \longleftrightarrow x_{min} = -\frac{2}{3}$$

Then, using the derivative in $x_{n+1} = x_n - \eta f'(x_n)$ we obtain:

$$x_{n+1} = x_n - \eta(6x_n + 4) = x_n - 6\eta x_n - 4\eta = x_n(1 - 6\eta) - 4\eta$$

At this point, if we subtract from both sides the real minimum x_{min} we ends up with the error at the iteration $n + 1$ on the left size:

$$x_{n+1} = x_n(1-6\eta)-4\eta \longleftrightarrow x_{n+1}-x_{min} = x_n(1-6\eta)-4\eta-x_{min} \longleftrightarrow e_{n+1} = x_n(1-6\eta)-4\eta-x_{min}$$

Here using the inverse formula of the error ($e_n = x_n - x_{min} \longleftrightarrow x_n = e_n + x_{min}$) we can write:

$$e_{n+1} = (e_n + x_{min})(1 - 6\eta) - 4\eta - x_{min}$$

which we can develop as:

$$e_{n+1} = e_n + x_{min} - 6\eta e_n - 6\eta x_{min} - 4\eta - x_{min} = e_n(1 - 6\eta) - \eta(6x_{min} + 4)$$

But we already know that for x_{min}, $6x+4 = 0$, since is the value of x that satisfy $f'(x) = 0$. So we ends up with just:

$$e_{n+1} = e_n(1 - 6\eta)$$

As said before, to be sure that we're reaching the minimum, $|e_{n+1}| < |e_n|$ So we can surely say that, thanks to the condition above:

$$|e_n(1 - 6\eta)| < |e_n| \longleftrightarrow |e_n||(1 - 6\eta)| < |e_n|$$

Since the absolute value of error must be > 0 (because cannot be 0, in that case we would have already reached the minimum) we can divide by $|e_n|$:

$$|1 - 6\eta| < 1$$

which equal to say:

- $1 - 6\eta < 1$ which is solved for $\eta > 0$
- $1 - 6\eta > -1$ which is solved for $\eta < \frac{1}{3}$

So $0 < \eta < \frac{1}{3}$ So for values of $\eta \to 0$ there is no improvement because the rate at which the model learn, is too small, and so the error does not decrease. On the other hand, for values of $\eta > \frac{1}{3}$ we might ends up with an exploding gradients, basically leading us in a situation where the error instead of decrease, increase as the number of iteration grow.

3 Class separability in 2D

3.1 Point A

We start by plotting our two classes We clearly cannot find a linear classifier that distin-

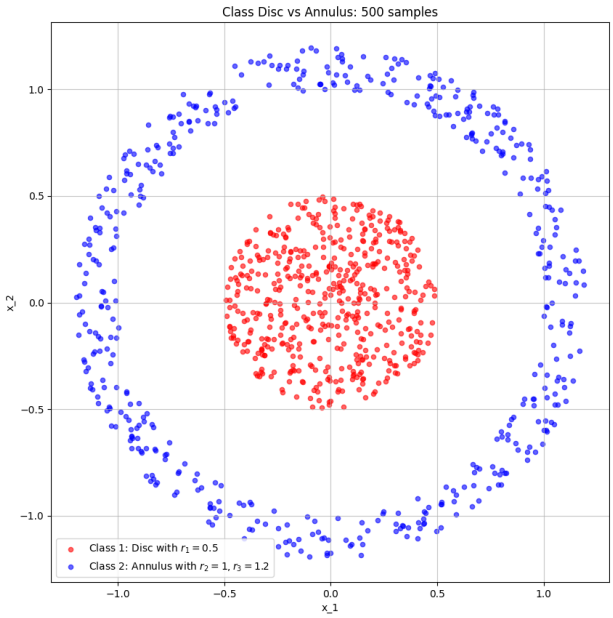

Figure 1

guish this classes since the classes are not linearly separable (we would need a classifier that looks like a ring to separate them). So practically, there is no possible lines that can separate them in 2D plane (one solution could be to find some features that can allows us to make classes separable in the feature space, maybe even using some kernel function)

3.2 Point B

Is it not possible to perfectly separate the two classes by using a single layer with two neurons for each of it we use as activation function a ReLU. In fact, since in this way we would obtain only two line, there is no way in which we can arrange these line to make them close in a figure that can perfectly distinguish the two classes, so the decision boundary generated would be open or in any case include also elements from the other class. For example we can obtain a reversed cone with the vertex below the first class (the red one) but from which start two lines that will never meet again (basically to close the decision boundary before the second class start). The other option would be a separator that create a band that can completely contains the first class, but that also contains the points from the second class (look the figure 2).

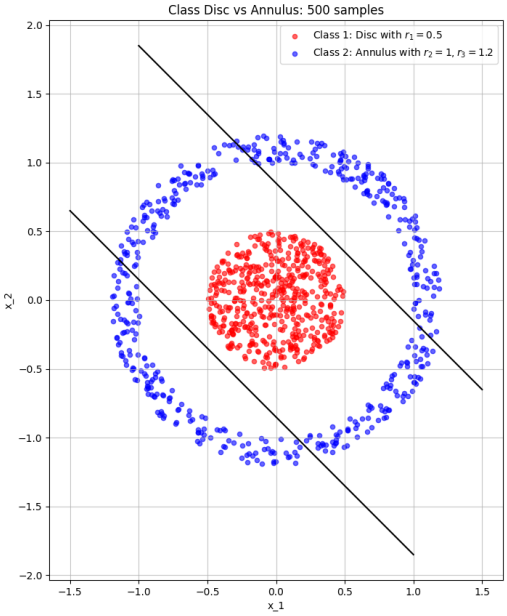

Figure 2

3.3 Point C

The activation function that we might need must be able to create a circle around the first (red) class. We remember that the equation of the circumference with center in the origin is:

$$x^2 + y^2 = r^2$$

So a possible solution could be an activation function that take the square of the input, for example:

$$f(x) = x^2$$

In that case, we could imagine this values for that first layer (the hidden one):

$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \qquad \text{and} \qquad \mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

In this way we can write the output of the two neurons as:

$$z_1^{(1)} = f(\mathbf{w}^{(1)}\mathbf{x} + b_1^{(1)}) = f(1 \cdot x_1 + 0 \cdot x_2 + 0) = f(x_1) = x_1^2$$

and in the same way

$$z_2^{(1)} = f(\mathbf{w}^{(1)}\mathbf{x} + b_2^{(1)}) = f(0 \cdot x_1 + 1 \cdot x_2 + 0) = f(x_2) = x_2^2$$

so that defining these values as next input that must multiply weights for the final layer (the output one):

$$\mathbf{W}^{(2)} = \begin{bmatrix} 1 & 1 \end{bmatrix} \qquad \text{and} \qquad b^{(2)} = -r^2$$

we arrive to:

$$y = \mathbf{W}^{(2)}\mathbf{z}^{(1)} + b^{(2)} = 1 \cdot z_1^{(1)} + 1 \cdot z_2^{(1)} - r^2 = x_1^2 + x_2^2 - r^2$$

That is exactly the equation of the circumference centered in zero.
For instance, we can show that for $r = 0.75$ we can get a separator for the two classes as shown below:

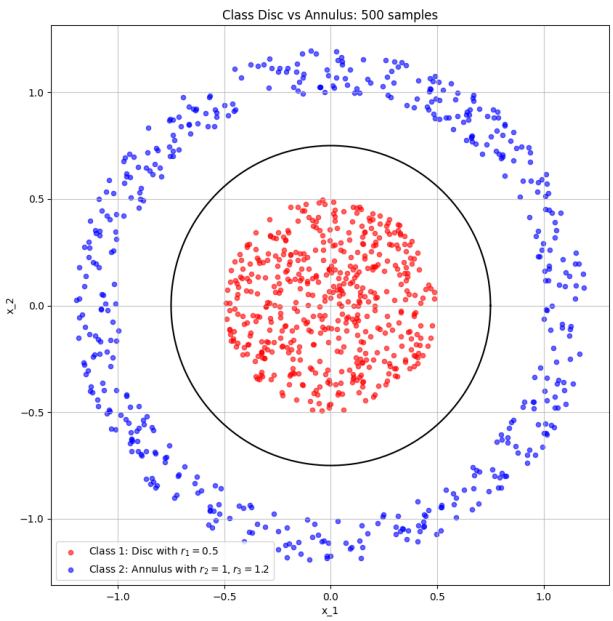

Figure 3

3.4 Point D

The minimum number of ReLU that we need to successfully separate the two classes are three. In fact, in that we case we can combine the three possible ReLU so that each combination gives us a different linear function used to create the closed (convex) figure that act as decision boundary. The total (max) number of edges that the figure can have is six (as the possible combination of linear function that we can obtain combining the three ReLU), in fact we can use to create the decision boundary the separator generated by each one of the ReLU, as well as the separator generated by the combination of two of them. The final result so will be an hexagonal-based cone, for example cut by the $z = 0$ plane, so that for $z > 0$ we will have the points in the internal disc (basically the peak of the cone) and for $z < 0$ all the other points (basically the blue-class). So our decision boundary in this example case will be the the level curve for which the output $z = 0$.

3.5 Point E

If we say that the inner disc must have $r_1 < 1$ this means that we are searching an equation that can relate the number of edges of the polygon in which the disc is inscribed with the radius of the inner disc (r_1) knowing that this polygon must be circumscribed by a circle created by the internal of the annulus that have radius $r_2 = 1$. So in this situation, all

the vertices of the polygon stays in the circle with $r_2 = 1$ and the circle with the $r_1 < 1$ touch the middle of each edge of the polygon. Knowing that the formula of the apothem (that is exactly the radius of the circle inscribed in the polygon) is:

$$\alpha = R\cos(\frac{\pi}{n})$$

where R is the radius of the circle circumscribed to the polygon (that we know is equal to $r_2 = 1$) we can simply derive the inverse formula that connect the number of the edges of the polygon to apothem (basically r_1)

$$\alpha = \cos(\frac{\pi}{n})$$

but this is the extreme case where we have that the circle inscribed in the polygon is exactly the inner circle: basically a point(s) of the class touch one (or more) edge(s). So we hope to find a polygon solution that have an higher apothem (with radius of the circle circumscribing the polygon still lower than the radius of the internal circle of the annulus, that in this case is 1):

$$\cos(\frac{\pi}{n}) \geq r_1$$

so that the relation between the number of edges and the radius of the inner disc is given by:

$$n \geq \frac{\pi}{\arccos(r_1)}$$

and the minimum number of n is given by:

$$n = \lceil \frac{\pi}{\arccos(r_1)} \rceil$$

So, when we have a value of r_1 that increase to 1, basically the number of edges increase as well, since we need a more complicate polygon to approximate the circle that act as decision boundary.
e.g.

$$\lim_{r_1 \to 1} = \frac{\pi}{\arccos(r_1)} = \frac{\pi}{0^+} = +\infty$$

It must be mentioned that to be precise, the number that we have found is the number of edges that the polygon must have, not the number of ReLU. As seen in the point D, we can create a more complex figure (in the case above an hexagon) even if the number of ReLU used is smaller then the number of edges (e.g. above we have used only three ReLU). In fact[2], each ReLU define an half-plane, and by combining two near ReLU, in the region where both of them are active we have a "deviation" of the half-plane created by the single ReLU (basically we have a vertex) so that, an edge is created, that finish (with another vertex) when only the second ReLU remains active. So, each ReLU can generate (max) two edges in the polygon, so that the number of ReLU needed is:

$$\text{Number of ReLU} \approx \frac{n}{2}$$

[2]Note: The things said regarding the way in which ReLU generate the figure, are connected to the specific problem of classification that we are discussing